Offline Implementation Plan

Running the Voice-Controlled KUKA KR4 R600 Without a Cloud Server

Supplementary section to the main report — Voice-Based Natural-Language Control of a KUKA KR4 R600 using the OpenGLRobot Simulator and Python. This document describes, step by step, how the existing system can be converted from cloud-based Google Web Speech recognition to a fully offline, on-device pipeline.

1 Why Move Offline

The current application sends every spoken command as audio to the Google Web Speech endpoint and waits for a transcription to return. This works, but it carries three limitations identified in the evaluation of the main report: it depends on a live internet connection, it adds a network round-trip to every command, and it offers no control over how the recogniser interprets the small, fixed command vocabulary. In an industrial cell — often isolated from the public internet for security reasons — none of these is acceptable for a production deployment.

An offline recogniser removes all three problems at once. The audio never leaves the machine, so there is no network latency and no connectivity requirement, and because the engine runs locally it can be constrained to recognise only the project's command words — which directly improves accuracy on the short, ambiguous tokens ("world", "x", "speed") that the statistics section flagged as unreliable. Privacy, determinism and offline operation are gained together.

1.1 What Changes and What Stays

The conversion is deliberately surgical. Only the speech-capture-and-transcribe stage changes; every other part of the system — the COM bridge to OpenGLRobot, the command parsers, the mode state machine, the Tkinter dashboard and the keyboard fallback — is untouched.

Component	Now (online)	After (offline)
Audio capture	sounddevice, 3 s fixed WAV	sounddevice / PyAudio 16 kHz stream
Speech-to-text	Google Web Speech (cloud)	Vosk (local Kaldi model)
Vocabulary control	None — open dictation	Grammar limited to command words
Network needed	Yes	No
Robot control (COM)	win32com → OpenGLRobot	Unchanged
Parsers / state machine	normalize + parse + modes	Unchanged
GUI + keyboard fallback	Tkinter dashboard	Unchanged

Table 1 — Scope of the offline conversion. Only two rows change.

2 Choosing the Offline Engine

Two mature offline engines are realistic candidates: Vosk (built on the Kaldi toolkit) and a local build of Whisper (for example faster-whisper). For this project the choice is clear, and it is Vosk.

- **Vosk** is lightweight, runs in real time on a plain CPU, and ships small per-language models of roughly 40–50 MB. Crucially, it offers a **streaming API** and a **grammar / limited-vocabulary mode** in which the recogniser is told the exact list of phrases to expect. For a fixed command set like this one, that single feature is what lifts accuracy on the problem words.
- **Whisper (local)** is more accurate on free-form, noisy or accented speech, but it is heavier: it generally wants a GPU for low latency, the useful models are far larger, and on CPU the record-then-transcribe cycle is slow. It also has no closed-grammar mode, so it cannot be constrained to the command vocabulary.

Because this application has a **small, known, closed vocabulary** and must run responsively on an ordinary lab PC without a GPU, Vosk is the better engineering fit. Whisper would be the choice only if the system later needed open-ended dictation.

> **Recommendation:** Use Vosk with the small English model (`vosk-model-small-en-us`, ~40 MB) in grammar-constrained mode, seeded with the exact command vocabulary from Table 1 of the main report. Keep the German model available for the bilingual wake word.

3 Step-by-Step Implementation Plan

The following seven steps take the project from the current online build to a fully offline one. They are ordered so that the system keeps working after every step and can be tested incrementally.

Step 1 — Install the offline dependencies

Add Vosk and a microphone-stream library to the environment. PyAudio gives a continuous 16 kHz stream that suits Vosk better than the current fixed-length recording, though the existing sounddevice capture can also be reused.

```
pip install vosk
pip install pyaudio        # or keep sounddevice

# pywin32, tkinter etc. stay exactly as they are
```

No internet account, API key or FLAC encoder is needed any more — the Google path required all three.

Step 2 — Download and place the model

Download the small English model from the Vosk model page, unzip it, and place the folder beside the script. This ~40 MB folder is the entire “server” — it lives on disk and is loaded once at start-up. Optionally add the small German model so the German wake word “hallo” is recognised offline too.

```
D:\VCRoboticsProj\OpenGLRobot\
  +-- kuka_voice_control_v8.py
  +-- model\                     <- vosk-model-small-en-us-0.15 (renamed)
  +-- model-de\                  <- optional German model
```

Because the model is a local file, it should be committed to the project archive (or its download documented) so the system is reproducible on another machine — this also fixes the per-machine path friction noted in the main report.

Step 3 — Build the command grammar

This is the step that gives the offline version its accuracy advantage. Instead of open-vocabulary dictation, Vosk is handed the explicit list of words and short phrases the system accepts. The recogniser will then only ever return one of these, so the ambiguous short tokens can no longer be mis-heard as unrelated English words. The list comes straight from the command reference already in the code.

```
import json
COMMAND_PHRASES = [
    "hello", "hallo", "halt", "stop", "home", "switch", "change",
    "world", "base", "axis", "tcp",
    "forward", "backward", "back", "right", "left", "up", "down",
    "base one", "base two", "tool one", "tool two",
    "axis one", "axis two", "axis three",
    "axis four", "axis five", "axis six",
    "clockwise", "anticlockwise",
    "coordinate", "speed", "motion",
    "low", "mid", "high", "lin", "ptp",
    "[unk]"   # lets the engine reject anything off-vocabulary
]
GRAMMAR = json.dumps(COMMAND_PHRASES)
```

The trailing `[unk]` token is important: it gives the recogniser an explicit “none of these” option so that random speech is rejected rather than forced onto the nearest command.

Step 4 — Replace the listen function

All of the speech logic in the project is contained in one function, `listen_once_normalized()`. Only the second half of it — the recording and the call to `recognize_google()` — is replaced. The Priority 1 fast-path (GUI buttons and the manual text box reading from `cmd_queue`) stays exactly as it is, so the keyboard fallback is unchanged.

```
from vosk import Model, KaldiRecognizer
import pyaudio, json

model = Model("model")                # load once at start-up

def listen_vosk(timeout=4.0):
    rec = KaldiRecognizer(model, 16000, GRAMMAR)   # grammar-limited
    p = pyaudio.PyAudio()
    stream = p.open(format=pyaudio.paInt16, channels=1,
                    rate=16000, input=True, frames_per_buffer=4000)
    stream.start_stream()
    if app: app.set_mic(True)
    try:
        while True:
            data = stream.read(4000, exception_on_overflow=False)
            if rec.AcceptWaveform(data):
                text = json.loads(rec.Result()).get("text", "")
                if text.strip():
                    return normalize_command(text)
    finally:
        stream.stop_stream(); stream.close(); p.terminate()
        if app: app.set_mic(False)
```

The existing `normalize_command()` still runs on the result, so all the synonym handling continues to work. The phonetic-variant tables built for the Google mis-hearings can be trimmed, since the grammar prevents most of them — but leaving them in does no harm.

Step 5 — Wire it into the thread loop

In `listen_once_normalized()`, swap the old microphone block for a call to the new function. The change is a single line; everything downstream — the parsers, `handle_direct_cmd()`, the four mode loops and `robot_thread_main()` — receives the same normalised strings it always did and needs no edits.

```
def listen_once_normalized() -> str:
    # Priority 1: GUI / manual text (unchanged, instant, offline)
    try:
        return cmd_queue.get_nowait()
    except queue.Empty:
        pass
    # Priority 2: OFFLINE Vosk instead of Google STT
    return listen_vosk()
```

Step 6 — Use streaming to cut latency

The online version always waited a fixed three seconds before it could even begin recognising. Vosk processes audio as it arrives and returns a result the moment a complete phrase is detected, so a short command like “halt” is acted on almost immediately instead of after a fixed window plus a network round-trip. To keep the system from listening forever when no one speaks, add a simple silence or wall-clock timeout and fall back to returning an empty string — exactly the value the mode loops already treat as “nothing heard”.

- Return as soon as `AcceptWaveform` reports a full phrase (already handled above).
- Add a maximum-duration guard so an idle microphone does not block the loop.
- Optionally use `rec.PartialResult()` to show live text in the Voice Feedback panel as the operator speaks.

Step 7 — Test, tune and validate

Re-run the same per-command trials used in the statistics section and compare. The expected outcome is that the four problem commands move back inside the normal band on both accuracy and time, because the grammar removes the mis-hearings and the streaming removes the fixed delay.

1. Disconnect the machine from the internet entirely — the system must still work. This is the definitive proof of offline operation.
2. Verify each command in Table 1 of the main report is recognised; adjust the `COMMAND_PHRASES` list for any that still misfire.
3. Confirm the bilingual wake word: load the German model for “hallo” or add it to the English grammar.
4. Measure recognition time again and update the statistics figures with the offline numbers for a before/after comparison.
5. Check CPU load during continuous listening; if it is high on the lab PC, the small model and a slightly larger audio buffer keep it comfortable.

4 Expected Outcome and Trade-offs

After these steps the application performs the full workflow — wake word, coordinate-frame selection, jogging, axis rotation, tool selection, speed and motion changes, home and halt — with no internet

connection at any point. The benefits and the costs are summarised below so the trade-off is explicit.

4.1 Benefits

- **True offline operation.** No connectivity requirement; suitable for an isolated industrial cell.
- **Lower, more predictable latency.** The network round-trip and the fixed three-second window both disappear.
- **Higher accuracy on the weak commands.** The closed grammar prevents short tokens from being mis-heard as other English words.
- **Privacy.** Audio never leaves the machine.
- **No recurring cost or quota.** The Google endpoint is replaced by a local file.

4.2 Trade-offs to acknowledge

- **Disk and memory.** The model adds ~40 MB on disk and a modest RAM footprint when loaded.
- **CPU use while listening.** Recognition now runs locally, so it uses CPU that the cloud previously absorbed — negligible with the small model on a normal PC.
- **Vocabulary is fixed at build time.** New commands must be added to the `COMMAND_PHRASES` list; this is a feature for safety but means the grammar must be maintained alongside the parsers.
- **Slightly lower free-form accuracy.** Vosk's small model is less capable than the cloud on open speech — irrelevant here because the vocabulary is closed, but worth noting if open dictation is ever added.

4.3 Summary

Converting the system to run offline is a small, contained change: two new dependencies, one downloaded model, one rewritten listen function and one grammar list. The COM interface, the control logic and the entire GUI are untouched. The result is a more robust, more private and more responsive system that satisfies the offline requirement and, as a direct side effect, repairs the recognition weaknesses identified in the statistics section. Should fully open-ended natural-language commands ever be required, a local Whisper build is the natural next upgrade, at the cost of heavier hardware.

Document prepared as a supplementary planning section for the scientific project "Controlling an Industrial Robot in a simulated environment with Speech", RWU M.Sc. Mechatronics, Summer Semester 2026.